

SOAL TUGAS PRAKTIKUM P14

Penyusunan Laporan Analisis dan Refactoring Kode Aplikasi Web

Studi kasus diarahkan pada aplikasi web tugas besar

Komponen	Keterangan
Mata Kuliah	Rekayasa Perangkat Lunak 2
Pertemuan	P14
Topik	MVC, SOLID, Clean Code, High Cohesion, Low Coupling, dan Refactoring
Bentuk Tugas	Penyusunan laporan teknis berbasis analisis kode
Output Utama	Laporan DOCX/PDF dan bukti pendukung sesuai ketentuan dosen

Konteks Tugas

Mahasiswa diminta menyusun laporan analisis dan refactoring kode aplikasi web sesuai kelompok tugas besar (tubes) masing-masing. Laporan harus menunjukkan kemampuan membaca struktur aplikasi, menemukan masalah desain kode, merancang perbaikan, membuat diagram class sebelum dan sesudah refactoring, serta membuktikan aplikasi tetap berjalan.

1. Dokumen Acuan dan Fokus Tugas

Tugas ini disusun berdasarkan contoh laporan yang sudah dibuat sebelumnya, yaitu laporan analisis dan refactoring Portal Kuesioner Psikometrik. Mahasiswa tidak diminta menyalin isi contoh laporan, tetapi mengikuti pola berpikir, urutan analisis, dan bentuk bukti yang digunakan pada contoh tersebut.

Struktur laporan mahasiswa harus mengikuti 16 bagian utama pada contoh laporan, mulai dari Identitas Proyek sampai Lampiran. Bagian 9 dan 10 wajib memuat class diagram sebelum dan sesudah refactoring yang dibuat dari kode Graphviz DOT.

Acuan pada contoh laporan	Yang harus dipahami mahasiswa
Identitas proyek dan deskripsi aplikasi	Laporan harus dimulai dari konteks aplikasi yang dianalisis, bukan langsung membahas teori.
Struktur folder dan arsitektur MVC	Mahasiswa harus membaca repository lalu memetakan controller, model, view, route, database, dan helper/service jika ada.
Daftar 5 temuan masalah kode	Mahasiswa wajib menemukan minimal 5 masalah nyata atau realistis berdasarkan file/method yang diperiksa.
Before-after refactoring	Setiap temuan harus memiliki kode sebelum, masalah, prinsip terkait, strategi perbaikan, kode sesudah, dan dampak.

Acuan pada contoh laporan	Yang harus dipahami mahasiswa
Class diagram Graphviz	Diagram sebelum dan sesudah refactoring dibuat dengan kode DOT, dirender menjadi gambar, lalu ditempatkan pada bagian narasi yang relevan.
Analisis SOLID, Clean Code, Cohesion, Coupling	Analisis harus dikaitkan dengan temuan kode, bukan berupa ringkasan teori umum.
Bukti aplikasi tetap berjalan	Mahasiswa harus menyertakan tabel uji, screenshot, lint/test, endpoint, atau log yang menunjukkan fitur utama tetap aman.

Inti yang Dinilai

Laporan yang baik menunjukkan hubungan langsung antara kode yang diperiksa, masalah desain yang ditemukan, rancangan refactoring yang diusulkan, diagram sebelum-sesudah, dan bukti bahwa aplikasi tidak rusak.

2. Tujuan Pembelajaran

- 1) Mahasiswa mampu menjelaskan struktur aplikasi web berbasis MVC berdasarkan repository yang dianalisis.
- 2) Mahasiswa mampu mengidentifikasi masalah kode berdasarkan prinsip SOLID, Clean Code, High Cohesion, dan Low Coupling.
- 3) Mahasiswa mampu menyusun contoh refactoring before-after yang relevan dengan kode aktual.
- 4) Mahasiswa mampu membuat class diagram sebelum dan sesudah refactoring menggunakan Graphviz DOT dan gambar hasil render.
- 5) Mahasiswa mampu menyusun laporan teknis akademik yang sistematis, berbasis bukti, dan dapat diuji ulang.

3. Skenario Tugas

Anda bertindak sebagai analis kode dan dokumentator teknis. Anda memperoleh sebuah aplikasi web yang sudah berjalan. Tugas Anda bukan membuat aplikasi baru, tetapi menyusun laporan analisis dan rancangan refactoring terhadap aplikasi tersebut. Gunakan aplikasi web yang ditentukan dosen atau aplikasi kelompok masing-masing. Jika dosen menyediakan aplikasi contoh, gunakan aplikasi tersebut sebagai objek analisis.

Contoh acuan laporan dapat dilihat pada dokumen contoh laporan refactoring Portal Kuesioner Psikometrik. Acuan tersebut digunakan untuk memahami format, kedalaman analisis, dan cara menyajikan bukti, bukan untuk disalin mentah.

4. Ketentuan Umum

- 1) Kerjakan secara kelompok sesuai pembagian kelas.

- 2) Gunakan repository/salinan kerja yang aman. Jangan mengubah aplikasi produksi atau artefak tubes aktif.
- 3) Mahasiswa tidak diwajibkan mengubah kode sumber aplikasi utama; contoh before-after dapat berupa rancangan refactoring atau implementasi pada branch/salinan latihan.
- 4) Jika melakukan refactoring kode, lakukan pada branch atau salinan latihan, bukan pada sumber utama yang masih digunakan.
- 5) Analisis harus berdasarkan file, class, method, route, database, atau modul yang benar-benar ditemukan.
- 6) Jangan mengklaim fitur yang tidak ada pada repository. Jika bagian tertentu bersifat asumsi, beri catatan eksplisit.
- 7) Setiap temuan wajib dilengkapi lokasi kode, potongan kode sebelum refactoring, strategi refactoring, potongan kode sesudah refactoring, dan dampaknya.
- 8) Class diagram wajib menggunakan Graphviz DOT serta disertai gambar hasil render di dalam laporan.
- 9) Laporan wajib memuat bukti bahwa fitur utama tetap berjalan setelah refactoring atau setelah simulasi refactoring pada branch latihan.

5. Alur Kerja yang Dianjurkan

Langkah	Aktivitas	Output Sementara
1	Baca README, struktur folder, route/entry point, controller, model, view, dan skema database.	Catatan struktur aplikasi dan daftar file penting.
2	Pilih minimal 5 modul/method yang paling layak dianalisis.	Tabel ruang lingkup analisis kode.
3	Cari masalah kode pada modul yang dipilih.	Daftar temuan: lokasi, masalah, prinsip terkait, dampak.
4	Tulis contoh refactoring before-after untuk setiap temuan.	Potongan kode sebelum dan sesudah refactoring.
5	Buat class diagram sebelum refactoring.	File DOT dan gambar diagram yang menunjukkan masalah struktur awal.
6	Buat class diagram sesudah refactoring.	File DOT dan gambar diagram yang menunjukkan rancangan controller-service-repository-validator.
7	Hubungkan temuan dengan SOLID, Clean Code, High Cohesion, dan Low Coupling.	Tabel dan narasi analisis prinsip desain.
8	Uji atau dokumentasikan bukti aplikasi tetap berjalan.	Tabel uji, screenshot, log command, lint/test, atau endpoint.
9	Susun laporan final sesuai struktur wajib.	Dokumen laporan siap dikumpulkan.

6. Tugas yang Harus Dikerjakan

No	Bagian Tugas	Instruksi
1	Identifikasi aplikasi	Jelaskan nama aplikasi, tujuan, teknologi, framework/pola arsitektur, dan fitur utama.
2	Pemetaan struktur kode	Tampilkan struktur folder aktual dan jelaskan peran controller, model, view, route, service/helper, repository, atau database.
3	Analisis MVC	Jelaskan bagaimana request mengalir dari route/controller menuju model dan view.
4	Temuan masalah kode	Temukan minimal 5 masalah kode yang relevan, misalnya controller terlalu gemuk, query tersebar, duplikasi validasi, magic value, atau nama method kurang jelas.
5	Before-after refactoring	Untuk setiap temuan, sertakan potongan kode sebelum dan sesudah refactoring serta alasan perbaikannya.
6	Class diagram	Buat class diagram sebelum dan sesudah refactoring menggunakan Graphviz DOT, lalu masukkan gambar hasil render ke laporan.
7	Analisis prinsip desain	Bahas penerapan SOLID, Clean Code, High Cohesion, dan Low Coupling berdasarkan temuan kode.
8	Bukti aplikasi berjalan	Sertakan hasil pengujian, screenshot, log, lint, atau bukti endpoint yang menunjukkan fitur utama tetap berjalan.
9	Kesimpulan	Simpulkan dampak refactoring terhadap maintainability, readability, testability, dan risiko perubahan.

7. Struktur Laporan Wajib

Gunakan 16 bagian berikut agar urutan laporan mahasiswa konsisten dengan contoh laporan Portal Kuesioner Psikometrik.

No	Bab/Bagian Laporan	Isi Minimal
1	Identitas Proyek	Nama aplikasi, jenis aplikasi, topik, anggota, repository, tanggal.
2	Deskripsi Singkat Aplikasi	Tujuan aplikasi, pengguna, fitur utama, dan batasan analisis.
3	Tujuan Refactoring	Alasan refactoring dan sasaran kualitas kode.
4	Ruang Lingkup Analisis Kode	Minimal 5 modul/file/method yang dianalisis.
5	Struktur Folder Aplikasi	Struktur aktual, bukan struktur asumsi.
6	Ringkasan Arsitektur MVC	Pemetaan controller, model, view, route, database, helper/service.
7	Daftar Temuan Masalah Kode	Tabel temuan, prinsip terkait, dan dampak negatif.
8	Analisis Before-After Refactoring	Minimal 5 temuan, setiap temuan memuat kode sebelum-sesudah.
9	Class Diagram Sebelum	Kode Graphviz DOT dan gambar diagram.

No	Bab/Bagian Laporan	Isi Minimal
	Refactoring	
10	Class Diagram Sesudah Refactoring	Kode Graphviz DOT dan gambar diagram.
11	Analisis SOLID	Bahas SRP, OCP, LSP, ISP, DIP secara jujur sesuai bukti kode.
12	Analisis Clean Code	Naming, small functions, duplication, magic value, separation of concerns.
13	High Cohesion dan Low Coupling	Bandingkan kondisi sebelum dan sesudah refactoring.
14	Bukti Aplikasi Tetap Berjalan	Tabel pengujian, screenshot, command, log, atau hasil endpoint.
15	Kesimpulan	Ringkasan manfaat dan batasan refactoring.
16	Lampiran	Link repository, branch, commit, screenshot, file DOT, dan bukti lain.

8. Ketentuan Khusus Graphviz

- 1) Gunakan format Graphviz DOT, bukan hanya gambar manual.
- 2) Sediakan kode DOT pada laporan atau lampiran, dan simpan file diagram dengan ekstensi .dot.
- 3) Render kode DOT menjadi gambar PNG/SVG, lalu sisipkan gambar tersebut pada narasi bagian class diagram.
- 4) Diagram sebelum refactoring harus menunjukkan masalah desain, misalnya controller terlalu banyak bergantung pada model/database.
- 5) Diagram sesudah refactoring harus menunjukkan pemisahan tanggung jawab, misalnya controller, service, repository, validator, dan presenter.
- 6) Nama class dalam diagram harus konsisten dengan class/method yang dibahas pada laporan.

Contoh Ringkas Graphviz DOT

```
digraph ClassDiagram {
    rankdir=LR;
    node [shape=record];
    Controller [label="{Controller|+ submit()}"];
    Service [label="{Service|+ process()}"];
    Controller -> Service;
}
```

9. Format Pengumpulan

Komponen	Ketentuan
Nama file laporan	KelompokXX_Laporan_Refactoring_NamaAplikasi.docx atau sesuai instruksi dosen
Format utama	DOCX; PDF boleh ditambahkan jika diminta
Lampiran kode	Branch repository, ZIP, atau link commit sebelum-sesudah refactoring
Lampiran diagram	File DOT dan gambar PNG/SVG class diagram

Komponen	Ketentuan
Lampiran bukti uji	Screenshot aplikasi, hasil lint/test, hasil endpoint, atau log command
Batas waktu	Sesuai yang dinyatakan pada edlink

10. Rubrik Penilaian

No	Aspek Penilaian	Bobot	Indikator
1	Kesesuaian konteks aplikasi	10	Deskripsi aplikasi, fitur, dan struktur kode sesuai repository.
2	Analisis MVC dan struktur folder	10	Pemetaan lapisan aplikasi jelas dan berbasis file aktual.
3	Temuan masalah kode	20	Minimal 5 temuan kuat, spesifik, dan terkait prinsip desain.
4	Before-after refactoring	20	Setiap temuan memiliki kode sebelum-sesudah dan alasan perbaikan.
5	Class diagram Graphviz	10	Kode DOT dan gambar sebelum-sesudah relevan serta mudah dibaca.
6	Analisis SOLID/Clean Code/Cohesion/Coupling	15	Analisis tidak hanya teori, tetapi dikaitkan dengan kode.
7	Bukti aplikasi tetap berjalan	10	Ada bukti uji fitur utama setelah refactoring/simulasi refactoring.
8	Kerapian laporan dan etika akademik	5	Bahasa formal, sitasi/bukti jelas, tidak asal klaim, tidak plagiarisme.

11. Checklist Sebelum Dikumpulkan

- 1) Laporan memiliki seluruh bagian wajib dari identitas proyek sampai lampiran.
- 2) Minimal ada 5 temuan masalah kode.
- 3) Setiap temuan memiliki kode sebelum dan sesudah refactoring.
- 4) Analisis SOLID dan Clean Code dikaitkan langsung dengan kode.
- 5) Class diagram dibuat dengan Graphviz DOT dan gambar diagram sudah disisipkan di laporan.
- 6) Ada bukti aplikasi tetap berjalan atau bukti hasil pengujian branch latihan.
- 7) Bagian class diagram, before-after, rubrik, dan bukti uji sudah selaras dengan contoh laporan acuan.
- 8) Tidak ada klaim fitur yang tidak ditemukan pada repository.
- 9) Nama file dan format pengumpulan sudah sesuai instruksi.

12. Catatan Kejujuran Akademik

Mahasiswa wajib menulis laporan berdasarkan hasil pemeriksaan sendiri terhadap repository kelompok tubes. Contoh laporan boleh digunakan sebagai acuan struktur dan kedalaman, tetapi isi temuan, kode, diagram, dan bukti pengujian harus disesuaikan dengan aplikasi yang dianalisis.